The Commons

A Mathematical Framework for Sovereign Resource Allocation

Abstract

We present a mathematical framework for resource allocation based on the Entity-Attribute-Value (EAV) paradigm with **observer recognition**. The system consists of entities with attributes, where special attributes create relational structure (membership), flow gradients (potentials), and allocation constraints. Each attribute has an associated observer who recognized it, creating a **distributed epistemic property attribution system**. We derive a Mutual Priority Flow Equation and prove that when weights represent net-benefit gradients toward goal realization, honest preference signaling is the unique optimal strategy. This makes strategic manipulation, vote trading, and Sybil attacks strictly suboptimal. The framework unifies categorization, allocation, and type hierarchies within a single structure, providing mathematical foundations for sovereign coordination systems.

1 The Entity-Attribute-Value Model

Definition 1 (The Commons)**.** *Let $\mathbb{E}$ be the universe of entities, $\mathbb{A}$ the universe of attributes, $\mathbb{V}$ the universe of values, $\mathbb{T}$ the universe of types, $\mathbb{O} \subseteq \mathbb{E}$ the set of observers, and $\mathbb{T}_{discrete}$ discrete time. The **Commons** is the quadruple:*

$$\mathcal{C} = \langle \mathcal{N}, \Pi, \Omega, \{\Delta_t\}_{t \in \mathbb{T}_{discrete}} \rangle$$

where:

- $\mathcal{N} \subseteq \mathbb{E}$ *is a finite set of entities*
- $\Pi : \mathcal{N} \to (\mathbb{A} \to \mathbb{V})$ *assigns each entity a mapping from attributes to values*
- $\Omega : \mathcal{N} \times \mathbb{A} \to \mathbb{O}$ *records which observer recognized each attribute*
- $\{\Delta_t\}_{t \in \mathbb{T}_{discrete}}$ *is the allocation history*
- $\mathbb{T}$ *is the universe of types, where $\mathcal{N} \subseteq \mathbb{T}$ (entities can serve as types)*

Remark 1 (Distributed Epistemic Property Attribution)**.** *The Commons uses a **distributed Entity-Attribute-Value (EAV) model with observer recognition**:*

- ***Entities*** *are the fundamental objects (nodes in traditional graph terminology)*
- ***Attributes*** *are property keys (e.g., "memberOf", "potentials", "reputation")*
- ***Values*** *are property values (sets of entities, lists of potentials, numbers, etc.)*

- ***Observers*** *are entities who recognize/assert attributes of other entities*

The observer function Ω *creates a* ***distributed epistemic system****:*

$$\Omega(e, a) = o \quad \textit{means} \quad \textit{"Observer } o \textit{ asserts that entity } e \textit{ has attribute } a\textit{"}$$

This enables:

- ***Perspective****: Different observers may recognize different attributes*
- ***Provenance****: Every attribute has a source (who recognized it)*
- ***Distribution****: No single authority on truth*
- ***Disagreement****: Multiple observers can have conflicting views*

This model is more general than traditional EAV, as it captures who knows what about whom*.*

1.1 Attribute Types

Remark 2 (Value Flexibility)**.** *Values in* $\mathbb{V}$ *can be of any type: primitives (text, numbers, booleans), structured data (lists, maps, records), or domain-specific types (potentials, entity references, etc.). The model imposes no restrictions on value types, allowing maximum flexibility for different use cases.*

Definition 2 (Attribute Recognition)**.** *An* ***attribute recognition*** *(or* ***assertion****) is a tuple:*

$$(o, e, a, v) \in \mathbb{O} \times \mathcal{N} \times \mathbb{A} \times \mathbb{V}$$

meaning "Observer o *recognizes that entity* e *has attribute* a *with value* v*".*

The system state is the set of all current recognitions:

$$\mathcal{R} = \{(o, e, a, v) \mid \Omega(e, a) = o \wedge \Pi(e)(a) = v\}$$

Remark 3 (Observer-Relative Truth)**.** *In this model, there is no objective truth about attributes — only* ***observer-relative assertions****. An entity may have different attribute values according to different observers:*

$$\Omega(e, a) = o_1 \wedge \Pi(e)(a) = v_1 \quad \textit{(according to } o_1\textit{)}$$

$$\Omega(e, a) = o_2 \wedge \Pi(e)(a) = v_2 \quad \textit{(according to } o_2\textit{)}$$

where $v_1 \neq v_2$ *represents disagreement.*

For implementation, we typically maintain a single "current view" per entity-attribute pair, with Ω *tracking the most recent or most trusted observer.*

1.2 The Membership Attribute

Definition 3 (Membership as Attribute)**.** *The* ***memberOf*** *attribute creates the relational structure. For entity $e \in \mathcal{N}$:*

$$\Pi(e)(\text{“}memberOf\text{”}) \in \mathbb{V}_{refs}$$

The membership relation E is ***emergent*** *from this attribute:*

$$E = \{(e_1, e_2) \in \mathcal{N} \times \mathcal{N} \mid e_2 \in \Pi(e_1)(\text{“}memberOf\text{”})\}$$

For $e_1, e_2 \in \mathcal{N}$, we write $e_1 \to e_2$ when $e_2 \in \Pi(e_1)(\text{“}memberOf\text{”})$, meaning “$e_1$ is a member of e_2”.

Definition 4 (Categories and Participants)**.** *For any entity $e \in \mathcal{N}$:*

$$Categories(e) = \Pi(e)(\text{“}memberOf\text{”}) \tag{1}$$

$$Participants(c) = \{e \in \mathcal{N} \mid c \in \Pi(e)(\text{“}memberOf\text{”})\} \tag{2}$$

Definition 5 (Transitive Closure)**.** *Let E^* be the reflexive transitive closure of the emergent relation E. Define:*

$$Ancestors(e) = \{a \in \mathcal{N} \mid (e, a) \in E^* \wedge e \neq a\} \tag{3}$$

$$Descendants(c) = \{d \in \mathcal{N} \mid (d, c) \in E^* \wedge d \neq c\} \tag{4}$$

These capture the full inheritance hierarchy.

Remark 4 (Graph as Emergent Structure)**.** *In the EAV model, the graph $G = \langle \mathcal{N}, E \rangle$ is not a primitive but an* ***emergent structure*** *derived from the memberOf attribute. This has profound implications:*

- *The relational structure is just another attribute, not special*
- *Entities can have arbitrary attributes beyond membership*
- *The model generalizes naturally to other relational patterns*
- *Constraints can uniformly reference any attribute*
- *Different observers may recognize different membership relations*

1.3 Observer-Based Queries

Definition 6 (Observer Query)**.** *For observer $o \in \mathbb{O}$, entity $e \in \mathcal{N}$, and attribute $a \in \mathbb{A}$, define:*

$$\Pi_o(e, a) = \begin{cases} \Pi(e)(a) & \text{if } \Omega(e, a) = o \\ \perp & \text{otherwise} \end{cases}$$

This returns the attribute value only if *observer o recognized it.*

Definition 7 (Observer Assertions)**.** *The set of all assertions by observer o is:*

$$Assertions(o) = \{(e, a, v) \mid \Omega(e, a) = o \wedge \Pi(e)(a) = v\}$$

This captures “what does observer o think about the world?”

Definition 8 (Entity Observers)**.** *The set of observers who have recognized attributes of entity e is:*

$$Observers(e) = \{o \in \mathbb{O} \mid \exists a \in \mathbb{A} : \Omega(e, a) = o\}$$

This captures "who has opinions about entity e?"

Remark 5 (Consensus and Conflict)**.** *When multiple observers have recognized the same attribute of an entity, we may need to aggregate their views. Common strategies include:*

- ***Most recent**: Use the observation with highest timestamp*
- ***Highest confidence**: Weight by observer confidence scores*
- ***Trust-weighted**: Weight by observer reputation/trust*
- ***Consensus**: Use majority value (for discrete attributes)*

For the purposes of allocation, we typically use a single "current view" per entity-attribute pair, with Ω tracking the authoritative observer for that view.

1.4 The Potentials Attribute

Definition 9 (Potential)**.** *A **potential** is a 4-tuple $p = (\tau, q, C, M)$ where:*

- *$\tau \in \mathbb{T}$ is the type (from the universe of types)*
- *$q \in \mathbb{R}^{\pm}$ is the signed magnitude*
- *$C \in \mathcal{C}$ is a constraint predicate*
- *M is metadata (a map of key-value pairs)*

*We say p is a **source** if $q > 0$ and a **sink** if $q < 0$.*

Since $\mathcal{N} \subseteq \mathbb{T}$, entities can serve as types (types-as-nodes), but types can also be abstract identifiers, strings, or other type representations not necessarily corresponding to entities in $\mathcal{N}$.

For a potential $p = (\tau, q, C, M)$, we use projection notation:

- *$\tau(p)$ denotes the type component*
- *$q(p)$ denotes the magnitude component*
- *$C(p)$ denotes the constraint component*
- *$M(p)$ denotes the metadata component*

***Metadata for Space-Time Matching**: The metadata M enables sophisticated matching beyond type compatibility. Common metadata includes:*

- ***Temporal**: time_zone, recurrence (daily/weekly/monthly/yearly), start_date, end_date, availability_window (hierarchical time patterns)*
- ***Spatial**: location_type, coordinates (latitude/longitude), city, country, online_link*

- ***Constraints***: *advance_notice_hours, booking_window_hours, mutual_agreement_required*
- ***Descriptive***: *name, unit, description*

Two potentials p_1 (source) and p_2 (sink) are ***compatible*** *if:*

1. *Type matches:* $\tau(p_1) = \tau(p_2)$
2. *Constraint satisfied:* $C(p_2)$ *accepts the source entity*
3. *Metadata compatible: A user-provided match function* $\phi(M(p_1), M(p_2))$ *returns true*

The match function ϕ can check arbitrary compatibility criteria, such as:

- *Time overlap (accounting for timezones and recurrence patterns)*
- *Location proximity or online availability*
- *Mutual agreement requirements*

Terminology: *For intuitive exposition, we refer to a potential's magnitude as its* ***capacity*** *when the potential is a source ($q > 0$) and as its* ***need*** *when the potential is a sink ($|q|$ where $q < 0$).*

Definition 10 (Potentials as Attribute)**.** *The* ***potentials*** *attribute stores flow gradients. For entity $e \in \mathcal{N}$:*

$$\Pi(e)(\text{“potentials”}) \in \mathbb{V}_{pots}$$

This is a list of potentials representing the entity's capacities (sources) and needs (sinks).

Remark 6 (Types from Universe $\mathbb{T}$)**.** *In this framework, types come from the universe $\mathbb{T}$, which includes entities as a special case ($\mathcal{N} \subseteq \mathbb{T}$). This means:*

Types can be entities *(types-as-nodes): When $\tau \in \mathcal{N}$, the type is an entity that may itself have:*

- *Categories (via memberOf attribute, enabling type hierarchies)*
- *Potentials (via potentials attribute, enabling meta-properties and flows)*
- *Participants (entities with this entity in their memberOf)*

Types can be abstract: *When $\tau \in \mathbb{T} \setminus \mathcal{N}$, the type is an abstract identifier (e.g., string, UUID) not corresponding to any entity in the system.*

This creates a flexible type system: use types-as-nodes when you want rich type metadata and hierarchies, use abstract types when you just need simple type matching. For example, an entity representing “MealType” could have “Food” and “Consumable” in its memberOf attribute, while also being referenced as the type in potentials like $(\text{MealType}, +100)$.

Definition 11 (Accessing Potentials)**.** *For entity $e \in \mathcal{N}$, we define:*

$$P(e) = \Pi(e)(\text{“potentials”})$$

This is the list of potentials for entity e. We can filter by type:

$$P(e, \tau) = \{p \in P(e) \mid \tau(p) = \tau\}$$

1.5 Constraints and Weights

Definition 12 (Constraint)**.** *A **constraint** is a function $C : \mathcal{N} \to \mathbb{R}_{\geq 0} \cup \{\infty\}$ that limits the maximum flow to each entity:*

- $C(e) = 0$ *excludes e from receiving flow*
- $C(e) = \infty$ *imposes no limit on e*
- $C(e) = k \in (0, \infty)$ *caps flow to e at k*

Constraints express policy limits, rate caps, distance bounds, and other restrictions on flow.

***Attribute-based constraints**: Constraints can reference any attribute:*

$$C_{reputation}(e) = \begin{cases} \infty & \text{if } \Pi(e)(\text{“reputation”}) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Definition 13 (Filter as Special Constraint)**.** *A **filter** is a constraint that takes only values in $\{0, \infty\}$:*

$$\Phi : \mathcal{N} \to \{0, \infty\} \subset \mathbb{R}_{\geq 0} \cup \{\infty\}$$

Filters determine binary eligibility (include/exclude) and are a special case of constraints.

Definition 14 (Weight)**.** *A **weight function** is $w : \mathcal{N} \to \mathbb{R}_{\geq 0}$ assigning preference to each entity.*

***Attribute-based weights**: Weights can reference any attribute:*

$$w_{reputation}(e) = \Pi(e)(\text{“reputation”})$$

1.6 Allocation Records

Definition 15 (Record)**.** *An **allocation record** at time t is:*

$$\Delta_t = (t, s, \tau, \{(r_i, q_i)\}_{i=1}^{n}, C, w, M)$$

where s is the source entity, τ the type, $\{(r_i, q_i)\}$ the recipient-quantity pairs, and M metadata.

2 Flow Equations: From Simple to Complete

Remark 7 (Conceptual Roadmap)**.** *This section develops the allocation mechanism through three progressive refinements:*

1. ***Mutual Priority Flow** (§4.1): Foundational equation respecting both provider and recipient priorities*
2. ***Full Utilization** (§4.2): Remainder redistribution for 100% capacity use*
3. ***Multi-Tier Cascades** (§4.3): Priority ordering across tiers*

Each refinement addresses a practical need, building toward a complete allocation mechanism that is voluntary, efficient, and flexible.

2.1 The Mutual Priority Flow Equation

Remark 8 (Sovereign Coordination Requires Dual Priorities)**.** *In sovereign coordination systems, both providers and recipients have autonomy over their preferences. A provider chooses how to distribute their capacity among recipients based on their priorities. Simultaneously, a recipient chooses which providers they wish to receive from based on trust, alignment, quality, or other factors.*

The allocation mechanism must respect ***both*** *parties' preferences. This is the essence of voluntary coordination: no allocation occurs without mutual agreement.*

Definition 16 (Recipient Prioritization)**.** *For recipient $r \in \mathcal{N}$ with need $q_r < 0$ of type τ, let S_r^τ be the set of sources:*

$$S_r^\tau = \{s \in \mathcal{N} \mid \exists p_s \in P(s) : \tau(p_s) = \tau \wedge q(p_s) > 0\}$$

The recipient's ***prioritization function*** *$G_r : S_r^\tau \to \mathbb{R}_{\geq 0}$ assigns weights to sources, normalized:*

$$\sum_{s \in S_r^\tau} G_r(s) = 1$$

The ***expectation*** *from source s is:*

$$E_{sr}^\tau = |q_r| \cdot G_r(s)$$

This represents what recipient r expects to receive from source s, based on their prioritization of that source.

Theorem 1 (Mutual Priority Flow Equation — The Foundational Allocation Formula)**.** *Let $s \in \mathcal{N}$ be a source entity with potential $p_s = (\tau, q_s, C_s)$ where $q_s > 0$. Let C be a constraint function and $R \subseteq \mathcal{N}$ be the set of eligible recipients:*

$$R = \{r \in \mathcal{N} \mid C(r) > 0 \wedge \exists p_r \in P(r) : \tau(p_r) = \tau \wedge q(p_r) < 0\}$$

The flow from source s to recipient $r \in R$ is:

$$\boxed{Flow(s, r, \tau) = \min\left(q_s \cdot \frac{w_s(r)}{\sum_{r' \in R} w_s(r')},\ E_{sr}^\tau,\ C(r)\right)}$$

This enforces four constraints:

1. ***Provider's priority****: Share proportional to provider's weight $w_s(r)$*
2. ***Recipient's priority****: Cannot exceed recipient's expectation $E_{sr}^\tau = |q_r| \cdot G_r(s)$*
3. ***Need bound****: $E_{sr}^\tau \leq |q_r|$ by construction*
4. ***Policy constraint****: Cannot exceed limit $C(r)$*

The min *operator finds the* ***optimal alignment point*** *between provider and recipient priorities, ensuring neither party's preferences are violated. This is the foundational allocation formula for the Commons framework.*

Proof. The flow is determined by four constraints:

1. **Provider proportionality**: Each recipient receives a share proportional to the provider's weight for them
2. **Recipient expectation**: No recipient receives more than they expect from this provider
3. **Need bound**: Recipient expectations are bounded by their total need
4. **Policy constraint**: External constraints limit maximum flow

The min function ensures all constraints are satisfied simultaneously:

- If provider wants to give more than recipient expects: allocation = expectation (respects recipient's priority)
- If recipient expects more than provider wants to give: allocation = provider's share (respects provider's priority)
- Neither party's priorities are violated

This is the unique point of maximum mutual agreement. □ □

Corollary 1 (Conservation)**.** *The total flow from source s satisfies:*

$$\sum_{r \in R} Flow(s, r, \tau) \leq q_s$$

with equality when $\sum_{r \in R} \min(E^{\tau}_{sr}, C(r)) \geq q_s$ (total constrained expectation exceeds capacity).

Proof. Each recipient receives at most their proportional share, expectation, or constraint limit. The sum cannot exceed the source capacity q_s. Equality holds when the source is fully depleted, which occurs when total constrained demand exceeds capacity. □ □

Example 1 (Kitchen and Two People)**.** *Consider a kitchen with 100 meal capacity, and two people:*

- ***Alice**: needs 20 meals, prioritizes kitchen 60%*
- ***Bob**: needs 30 meals, prioritizes kitchen 40%*
- ***Kitchen weights**: Alice 50%, Bob 50%*

***Calculation**:*

$$\begin{aligned}
\textit{Provider shares:} \quad & \textit{Alice: } 100 \times 0.5 = 50 \textit{ meals} \\
& \textit{Bob: } 100 \times 0.5 = 50 \textit{ meals} \\
\textit{Recipient expectations:} \quad & \textit{Alice: } 20 \times 0.6 = 12 \textit{ meals} \\
& \textit{Bob: } 30 \times 0.4 = 12 \textit{ meals} \\
\textit{Flow (min of both):} \quad & Flow(k, Alice) = \min(50, 12, \infty) = 12 \\
& Flow(k, Bob) = \min(50, 12, \infty) = 12
\end{aligned}$$

***Result**: Total allocated = 24 meals (24% utilization), Remainder = 76 meals*

***Observation**: The recipient expectations are the binding constraint here. Both Alice and Bob expect only 12 meals from the kitchen (despite needing more), so that's what they receive. The remainder will be redistributed in §4.2.*

Remark 9 (Intuition: The Min as Mutual Agreement)**.** *The* min *operator represents mutual agreement between provider and recipient:*

- ***Provider offers**: X based on their weights*
- ***Recipient expects**: Y based on their prioritization*
- ***Allocation**:* $\min(X, Y)$ *— neither can force more on the other*

How expectations work in practice:

***Single provider**: When a recipient has only one provider for a type, their expectation defaults to their full need ($E_{sr} = |q_r|$). The* min *operator then naturally caps allocation at the provider's proportional share.*

***Multiple providers**: Recipients distribute their expectations across providers based on prioritization: $E_{sr} = |q_r| \cdot G_r(s)$, where $\sum_s G_r(s) = 1$. This ensures expectations sum to total need and coordinates allocation across providers.*

This is the mathematical expression of voluntary coordination: allocation requires mutual consent, with expectations adapting to the number of available providers.

Remark 10 (Voluntary Coordination)**.** *The mutual priority constraint embodies the principle of **voluntary coordination**: no allocation occurs without mutual agreement between provider and recipient. This distinguishes the Commons from coercive allocation mechanisms where one party can force allocation on another.*

Remark 11 (Special Case: Uniform Recipient Expectations)**.** *When all recipients have uniform expectations (e.g., $G_r(s) = 1/|S_r^\tau|$ for all s), the mutual priority equation reduces to a simpler form where only provider priorities matter. However, in practice, recipients always have preferences, making the mutual priority formulation the natural general case.*

2.2 Achieving Full Capacity Utilization

Remark 12 (The Remainder Problem)**.** *The Mutual Priority Flow Equation can leave unused capacity when recipient expectations are lower than provider shares. For example, if a provider wants to allocate 50 units to a recipient, but the recipient only expects 30 units, the* min *operator allocates 30, leaving 20 units unused. To achieve 100% capacity utilization, we introduce **remainder redistribution**: unused capacity is redistributed to recipients who can accept more, while maintaining proportional fairness according to provider weights.*

Definition 17 (Allocation Remainder)**.** *After initial allocation using the Mutual Priority Flow Equation (Theorem 1), the **remainder** is:*

$$R_s^\tau = q_s - \sum_{r \in R} Flow(s, r, \tau)$$

This represents unused capacity when recipient expectations are lower than provider shares, or when recipients hit their need bounds before exhausting provider capacity.

Theorem 2 (Complete Flow Equation with Full Utilization)**.** *To achieve 100% capacity utilization, allocation proceeds in two phases:*

Phase 1: Initial Allocation (Mutual Priority Flow)

$$a_{sr}^{(0)} = \min\left(q_s \cdot \frac{w_s(r)}{\sum_{r' \in R} w_s(r')},\ E_{sr}^{\tau},\ C(r)\right)$$

Phase 2: Iterative Remainder Redistribution

While $R_s = q_s - \sum_r a_{sr} > 0$ *and eligible recipients exist:*

1. *Identify eligible recipients:* $R_{eligible} = \{r \in R \mid \sum_{s'} a_{s'r} < |q_r| \wedge C(r) > \sum_{s'} a_{s'r}\}$
2. *Redistribute remainder:*

$$\Delta a_{sr} = \min\left(R_s \cdot \frac{w_s(r)}{\sum_{r' \in R_{eligible}} w_s(r')},\ |q_r| - \sum_{s'} a_{s'r}\right)$$

3. *Update:* $a_{sr} \leftarrow a_{sr} + \Delta a_{sr}$

Guarantee*: If total constrained need exceeds capacity (*$\sum_{r \in R} \min(|q_r|, C(r)) \geq q_s$*), this achieves:*

$$\sum_{r \in R} a_{sr} = q_s \quad \textit{(100\% utilization)}$$

Proof. The initial allocation respects both provider and recipient priorities via the min operator. When expectations are properly distributed (single provider: $E_{sr} = |q_r|$; multiple providers: $\sum_s E_{sr} = |q_r|$), the initial allocation typically achieves high utilization.

Remainder occurs primarily from:

- **Divisibility constraints**: Fractional allocations rounded down
- **Constraint caps**: $C(r)$ limits below expectations
- **Suboptimal expectations**: Recipients haven't learned optimal $G_r(s)$ yet

The iterative redistribution maintains proportional fairness ($w_s(r)$) while respecting remaining capacity under expectations and constraints. The process terminates when either all capacity is allocated ($R_s = 0$) or no recipients can accept more ($R_{\text{eligible}} = \emptyset$). When total constrained expectations $\geq$ capacity, eligible recipients always exist until capacity is exhausted, ensuring $\sum_r a_{sr} = q_s$. □ □

Example 2 (Kitchen Example: Remainder Redistribution)**.** *Continuing from Example 1, we had 76 meals remaining after initial allocation.*

Scenario 1: Original needs (Alice 20, Bob 30)

- *Alice received 12, needs 20 → can accept 8 more*
- *Bob received 12, needs 30 → can accept 18 more*
- *Eligible recipients: both Alice and Bob*

- *Remainder redistribution:* $76 \times 0.5 = 38$ *to each*
- *But Alice can only accept 8 more: Alice gets +8, leaving 68 meals*
- *Second round: Bob gets* $\min(68, 18) = 18$
- ***Final**: Alice 20, Bob 30, remainder 50 meals*

Scenario 2: Higher needs (Alice 60, Bob 80)

- *Initial: Alice 12, Bob 12 (same as before)*
- *Eligible: Alice can accept 48 more, Bob can accept 68 more*
- *Round 1: Alice gets* $\min(76 \times 0.5, 48) = 38$*, Bob gets* $\min(76 \times 0.5, 68) = 38$
- *After round 1: Alice 50, Bob 50, remainder 0*
- ***Final**: Alice 50, Bob 50, 100% utilization achieved!*

Proposition 1 (Sparse Network Behavior)**.** *When total need is less than capacity ($\sum_{r \in R} |q_r| < q_s$), the system converges to:*

$$\sum_{r \in R} a_{sr} = \sum_{r \in R} |q_r| < q_s \quad \textit{(partial capacity utilization)}$$

*All recipients are fully satisfied ($\Delta_r^\tau = 0$ for all r), achieving **universal need satisfaction**, but some capacity remains unused.*

Proof. When all recipients reach $\Delta_r^\tau = 0$, they have received exactly their declared need $|q_r|$. At this point, $R_{\text{eligible}} = \emptyset$ since no recipient can accept more allocation. The remainder redistribution algorithm terminates with:

$$R_s = q_s - \sum_r |q_r| > 0$$

This is the correct equilibrium: forcing allocation beyond recipient needs would violate the need bound constraint. □ □

Remark 13 (Sparse Networks Are Stable)**.** *Sparse networks (low total need relative to capacity) are perfectly stable. The system doesn't "fail" when it can't achieve 100% capacity utilization — it achieves 100% need satisfaction instead. Unused capacity can be:*

- *Reallocated to other resource types*
- *Saved for future allocation rounds*
- *Offered to new recipients who join the network*

*The key insight: **The system optimizes for recipient satisfaction, not provider utilization**. Full capacity utilization is desirable but not required for equilibrium.*

Remark 14 (Efficiency and Fairness)**.** *Remainder redistribution ensures no capacity is wasted while maintaining proportional fairness according to provider weights. This achieves Pareto efficiency: no reallocation can improve one recipient without violating provider priorities or recipient constraints.*

2.3 Multi-Tier Priority Cascades

Theorem 3 (Multi-Tier Allocation)**.** *Let $\{T_1, T_2, \ldots, T_n\}$ be a sequence of tiers with priority $T_1 \succ T_2 \succ \cdots \succ T_n$, where each tier $T_i = (C_i, w_i)$ specifies a constraint and weight. Define the sequential allocation:*

$$Flow_i(s, r, \tau) = \begin{cases} \min\left(q_i \cdot \frac{w_i(r)}{\sum_{r' \in R_i} w_i(r')}, |q_r|, C_i(r)\right) & \text{if } r \in R_i \\ 0 & \text{otherwise} \end{cases}$$

where $q_i = q_{i-1} - \sum_{r \in R_{i-1}} Flow_{i-1}(s, r, \tau)$ is the remaining capacity after tier $i-1$, with $q_1 = q_s$, and $R_i = \{r \in \mathcal{N} \mid C_i(r) > 0\}$.

Proof. Each tier allocates from the remaining capacity after higher-priority tiers. The process terminates when either capacity is exhausted or all needs are satisfied. □ □

3 Multi-Provider Coordination and Convergence

Remark 15 (From Single to Multiple Providers)**.** *The flow equations in Section 4 describe allocation from a single source to multiple recipients. In practice, multiple independent providers may simultaneously allocate to the same recipients. This section addresses how the system coordinates across providers to achieve equilibrium without centralized control.*

The key insight: each provider independently computes allocations using the Mutual Priority Flow Equation (Theorem 1), while recipients aggregate allocations to compute their ***distance from satisfaction****. This distance acts as a coordination signal that guides all providers toward equilibrium through iterative adjustment.*

3.1 Distance as Coordination Signal

Definition 18 (Distance from Need and Multi-Provider Aggregation)**.** *For recipient r and type τ, let S_r^τ be the set of all sources providing type τ. Define:*

Total received allocation*:*

$$A_r^\tau = \sum_{s \in S_r^\tau} a_{sr}^\tau$$

This aggregates allocations from all providers.

Distance from need*:*

$$\Delta_r^\tau = N_r^\tau - A_r^\tau$$

where:

- $N_r^\tau = |q_r|$ *is the declared need of recipient r for type τ*
- A_r^τ *is the total allocated to r (sum across all providers)*
- $\Delta_r^\tau > 0$ *indicates under-allocation (undershoot)*
- $\Delta_r^\tau < 0$ *indicates over-allocation (overshoot)*
- $\Delta_r^\tau = 0$ *indicates perfect satisfaction (equilibrium)*

This distance acts as a ***potential difference****, analogous to voltage in electrical systems or pressure difference in fluid dynamics. The aggregation across providers enables decentralized coordination: each provider independently computes allocations, while recipients aggregate the total to determine their satisfaction state.*

3.2 Convergence to Equilibrium

Definition 19 (Distance-Based Adjustment)**.** *The allocation adjustment based on distance follows:*

$$a_{sr}^{\tau}(t+1) = \begin{cases} a_{sr}^{\tau}(t) - \alpha|d_{sr}(t)| & \text{if } \Delta_r^{\tau}(t) < 0 \text{ (retract from overshoot)} \\ a_{sr}^{\tau}(t) + \alpha|d_{sr}(t)| & \text{if } \Delta_r^{\tau}(t) > 0 \text{ (expand toward undershoot)} \\ a_{sr}^{\tau}(t) - \alpha \cdot d_{sr}(t) & \text{if } \Delta_r^{\tau}(t) = 0 \text{ (rebalance for alignment)} \end{cases}$$

where:

- $d_{sr}(t) = a_{sr}^{\tau}(t) - t_{sr}^{\tau}$ *is the deviation from target (computed using Theorem 1)*
- $\alpha \in (0, 1]$ *is the damping factor (typically 0.8)*
- t *is the discrete iteration number*

This implements discrete-time gradient descent on the energy landscape:

$$E(t) = \frac{1}{2} \sum_{r,\tau} (\Delta_r^{\tau}(t))^2$$

At each iteration, energy decreases:

$$E(t+1) \leq (1-\alpha)^2 E(t)$$

Remark 16 (Continuous-Time Interpretation)**.** *In the continuous-time limit (infinitesimal time steps $\Delta t \to 0$), the discrete update becomes:*

$$\frac{dE}{dt} = \lim_{\Delta t \to 0} \frac{E(t+\Delta t) - E(t)}{\Delta t} \leq -\alpha E(t)$$

This is analogous to thermodynamic systems where energy decreases monotonically toward equilibrium. However, the actual algorithm operates in discrete time with finite iteration steps.

Theorem 4 (Multi-Provider Equilibrium)**.** *In a system with multiple independent providers, the equilibrium state satisfies:*

$$\forall s, r, \tau : \quad a_{sr}^{\tau} = t_{sr}^{\tau}$$

where t_{sr}^{τ} is the target from Theorem 1 (with full utilization from Theorem 2).

At equilibrium:

1. ***Universal satisfaction*** *(if capacity sufficient):* $\Delta_r^{\tau} = 0 \; \forall r, \tau$
2. ***Full utilization*** *(if need sufficient):* $\sum_r a_{sr}^{\tau} = q_s \; \forall s, \tau$

3. ***Optimal alignment**: All allocations match targets respecting both provider and recipient priorities*

4. ***Pareto-optimal**: No reallocation can improve any participant without violating another's priorities*

Proof. Each provider independently computes targets using Theorem 1 and applies full utilization (Theorem 2). The distance-based adjustment ensures convergence to targets.

The system is a contraction mapping with rate $\kappa = (1 - \alpha) < 1$:

$$\|\Delta(t+1)\|_F \leq (1-\alpha)\|\Delta(t)\|_F$$

By the Banach fixed-point theorem, this converges to the unique fixed point. At equilibrium, $a^\tau_{sr} = t^\tau_{sr}$ for all providers, which implies:

- If $\sum_s q_s \geq \sum_r N^\tau_r$: sufficient capacity exists, so $\Delta^\tau_r = 0$ for all r
- If $\sum_r N^\tau_r \geq \sum_s q_s$: sufficient need exists, so full utilization ensures $\sum_r a^\tau_{sr} = q_s$ for all s
- All allocations respect both provider and recipient priorities via the min operator

□ □

Corollary 2 (Scarcity Equilibrium)**.** *When total capacity is insufficient ($\sum_s q_s < \sum_r N^\tau_r$), equilibrium is characterized by:*

$$a^\tau_{sr} = t^\tau_{sr} \quad \forall s, r, \tau \quad \textit{(allocations match targets)} \tag{5}$$

$$\sum_r a^\tau_{sr} = q_s \quad \forall s, \tau \quad \textit{(full capacity utilization)} \tag{6}$$

$$\Delta^\tau_r > 0 \quad \textit{for some } r \quad \textit{(partial satisfaction)} \tag{7}$$

At this equilibrium, no provider can improve any recipient's satisfaction without violating their own priority constraints or capacity limits. The distribution of scarce resources is determined by the interplay of all provider weights $w_s(r)$ and recipient prioritizations $G_r(s)$, achieving maximal satisfaction subject to capacity constraints.

Proof. When capacity is insufficient, not all needs can be satisfied. Full utilization ensures all capacity is used ($\sum_r a^\tau_{sr} = q_s$). The distance-based adjustment converges to the state where each provider's allocation matches their target, which represents their optimal distribution given their priorities. Some recipients remain unsatisfied ($\Delta^\tau_r > 0$), but the allocation is Pareto-optimal: any reallocation that improves one recipient would require violating some provider's priority weights. □ □

Remark 17 (The Utilization-Satisfaction Trade-off)**.** *The system exhibits a fundamental trade-off between capacity utilization and need satisfaction:*

***Abundance** (capacity > need):*

- *Need satisfaction: 100% (all recipients fully satisfied)*

- *Capacity utilization: $< 100\%$ (some capacity unused)*
- *Equilibrium: $\Delta_r = 0$ for all r, but $\sum_r a_{sr} < q_s$*

Scarcity *(capacity $<$ need):*

- *Need satisfaction: $< 100\%$ (some recipients unsatisfied)*
- *Capacity utilization: 100% (all capacity allocated)*
- *Equilibrium: $\sum_r a_{sr} = q_s$, but $\Delta_r > 0$ for some r*

System priority*: The algorithm prioritizes recipient satisfaction over provider utilization. Once a recipient is satisfied ($\Delta_r = 0$), they are removed from allocation even if capacity remains. The system will not force allocation on satisfied recipients merely to achieve 100% utilization.*

3.3 Implementation

Definition 20 (Iterative Allocation Algorithm)**.** ***Input****:*

- *Sources $S \subseteq \mathcal{N}$ with potentials $\{p_s = (\tau, q_s, C_s) \mid q_s > 0\}$*
- *Recipients $R \subseteq \mathcal{N}$ with potentials $\{p_r = (\tau, q_r, C_r) \mid q_r < 0\}$*
- *Provider weights $\{w_s : R \to \mathbb{R}_{\geq 0}\}_{s \in S}$ with $\sum_{r \in R} w_s(r) = 1$*
- *Recipient prioritizations $\{G_r : S \to \mathbb{R}_{\geq 0}\}_{r \in R}$ with $\sum_{s \in S} G_r(s) = 1$*
- *Damping factor $\alpha \in (0, 1]$ (typically 0.8)*
- *Convergence threshold $\epsilon > 0$ (typically 10^{-6})*

Output*: Allocations $\{a^\tau_{sr}\}$ at equilibrium*
Procedure*:*

1. ***Initialize****: Set $a^\tau_{sr} \leftarrow 0$ for all $s \in S, r \in R, \tau$*
2. ***Repeat until convergence****:*
   (a) ***Compute targets*** *for each provider s and recipient r:*

$$E^\tau_{sr} \leftarrow |q_r| \cdot G_r(s) \quad \textit{(recipient expectation)}$$
$$t^\tau_{sr} \leftarrow \min\left(q_s \cdot \frac{w_s(r)}{\sum_{r' \in R} w_s(r')},\, E^\tau_{sr},\, C(r)\right)$$

   (b) ***Apply remainder redistribution*** *(Theorem **??**):*

$$R_s \leftarrow q_s - \sum_{r \in R} t^\tau_{sr}$$
$$R_{eligible} \leftarrow \{r \in R \mid \sum_{s'} t_{s'r} < |q_r| \wedge C(r) > \sum_{s'} t_{s'r}\}$$
$$\textit{For } r \in R_{eligible}: \quad t^\tau_{sr} \leftarrow t^\tau_{sr} + \min\left(R_s \cdot \frac{w_s(r)}{\sum_{r' \in R_{eligible}} w_s(r')}, |q_r| - \sum_{s'} t_{s'r}\right)$$

(c) ***Compute distances*** *for each recipient* r*:*

$$A_r^\tau \leftarrow \sum_{s \in S} a_{sr}^\tau$$
$$\Delta_r^\tau \leftarrow |q_r| - A_r^\tau$$

(d) ***Adjust allocations*** *for each provider* s *and recipient* r*:*

$$d_{sr} \leftarrow a_{sr}^\tau - t_{sr}^\tau$$
$$a_{sr}^\tau \leftarrow \begin{cases} a_{sr}^\tau - \alpha|d_{sr}| & \text{if } \Delta_r^\tau < 0 \text{ (retract from overshoot)} \\ a_{sr}^\tau + \alpha|d_{sr}| & \text{if } \Delta_r^\tau > 0 \text{ (expand toward undershoot)} \\ a_{sr}^\tau - \alpha \cdot d_{sr} & \text{if } \Delta_r^\tau = 0 \text{ (rebalance for alignment)} \end{cases}$$

(e) ***Check convergence****:*

$$\text{if } \max_{r \in R, \tau} |\Delta_r^\tau| < \epsilon \text{ then break}$$

3. ***Return****:* $\{a_{sr}^\tau\}$

Theorem 5 (Convergence Rate)**.** *The iterative allocation algorithm (Definition 20) converges geometrically with rate* $\kappa = (1 - \alpha) < 1$*:*

$$\|\Delta(t+1)\|_F \leq (1 - \alpha)\|\Delta(t)\|_F$$

The number of iterations to reach error ϵ *from initial distance* $\|\Delta_0\|$ *is:*

$$T_\epsilon = O\left(\frac{\log(\epsilon/\|\Delta_0\|)}{\log(1 - \alpha)}\right)$$

For $\alpha = 0.8$*, error reduces by 99.99% in approximately 6 iterations.*

Proof. The distance-based adjustment implements gradient descent on the energy landscape:

$$E = \frac{1}{2} \sum_{r \in R, \tau} (\Delta_r^\tau)^2$$

Each iteration reduces this energy by factor $(1 - \alpha)$:

$$E(t+1) \leq (1 - \alpha)^2 E(t)$$

Therefore $\|\Delta(t+1)\|_F = \sqrt{2E(t+1)} \leq (1 - \alpha)\sqrt{2E(t)} = (1 - \alpha)\|\Delta(t)\|_F$.
Setting $\|\Delta(T)\|_F \leq \epsilon$ and solving:

$$(1 - \alpha)^T \|\Delta_0\|_F \leq \epsilon \implies T \geq \frac{\log(\epsilon/\|\Delta_0\|_F)}{\log(1 - \alpha)}$$

For $\alpha = 0.8$: $(1 - 0.8)^6 = 0.000064 \approx 0.0001$, giving 99.99% error reduction. □ □

Proposition 2 (Damping Factor Trade-offs)**.** *The damping factor* $\alpha \in (0, 1]$ *controls the convergence-stability trade-off:*

α	***Iterations to 99.99%***	***Stability***	***Use Case***
0.1	≈ 22	*Very High*	*Highly dynamic networks*
0.5	≈ 8	*High*	*Moderate dynamics*
0.8	≈ 6	*Good*	*Most scenarios (recommended)*
0.9	≈ 4	*Moderate*	*Static networks*
0.99	≈ 2	*Low*	*Rare, oscillation risk*

Optimal range: $\alpha \in [0.7, 0.9]$ *balances fast convergence with stability.*

Proof. From Theorem 5, iterations to reach error ϵ is $T_\epsilon = \log(\epsilon/\|\Delta_0\|)/\log(1-\alpha)$.

For 99.99% error reduction ($\epsilon/\|\Delta_0\| = 0.0001$):

- $\alpha = 0.1$: $T \approx \log(0.0001)/\log(0.9) \approx -9.21/-0.105 \approx 22$
- $\alpha = 0.5$: $T \approx -9.21/-0.693 \approx 8$
- $\alpha = 0.8$: $T \approx -9.21/-1.609 \approx 6$
- $\alpha = 0.9$: $T \approx -9.21/-2.303 \approx 4$

As α increases, convergence accelerates, but larger adjustment steps increase oscillation risk when distance estimates are noisy. □ □

Remark 18 (Practical Guidance)**.** *In practice, $\alpha = 0.8$ works well across diverse scenarios. For highly dynamic networks where needs/capacities change frequently, reduce to $\alpha = 0.5$ for better stability. For static networks with infrequent updates, increase to $\alpha = 0.9$ for faster convergence.*

3.4 Robustness to Network Partitions

Proposition 3 (Self-Correction via Negative Feedback)**.** *When network partitions create inconsistent views of A_r^τ, the system self-corrects through negative feedback:*

Temporary over-allocation *($\Delta_r^\tau < 0$): All providers retract allocations proportionally*

Temporary under-allocation *($\Delta_r^\tau > 0$): All providers expand allocations proportionally*

Errors decay geometrically with rate $(1-\alpha)^t$, ensuring rapid convergence once partitions heal.

Proof. The distance-based adjustment (step 2d in Definition 20) creates a negative feedback loop. When $\Delta_r^\tau < 0$ (over-allocation), all providers reduce a_{sr}^τ toward targets. When $\Delta_r^\tau > 0$ (under-allocation), all providers increase a_{sr}^τ toward targets. By Theorem 5, errors decay at rate $(1-\alpha)^t$. □ □

Proposition 4 (Bounded Oscillation)**.** *Maximum over-allocation is bounded by total capacity:*

$$A_r^\tau \leq \sum_{s \in S} q_s$$

Even in worst-case network partitions, the system remains bounded and converges once partitions heal.

Proof. Each provider's allocation is bounded by their capacity: $\sum_r a_{sr}^\tau \leq q_s$. Therefore total allocation to any recipient is bounded by total capacity: $A_r^\tau = \sum_s a_{sr}^\tau \leq \sum_s q_s$. The system cannot diverge unboundedly. □ □

3.5 Distributed Coordination Mechanisms

Remark 19 (The Distributed Challenge)**.** *In distributed systems, providers may not have consistent views of recipient distances Δ_r^τ. Network delays, partitions, and asynchronous updates create information inconsistencies. How does the system maintain convergence guarantees under these conditions?*

Definition 21 (Recipient Broadcast Protocol)**.** *Recipients broadcast their current distance Δ_r^τ to all providers:*

1. *Recipient computes: $\Delta_r^\tau = |q_r| - \sum_s a_{sr}^\tau$*
2. *Recipient broadcasts: $(r, \tau, \Delta_r^\tau, timestamp)$*
3. *Providers receive and cache latest Δ_r^τ value*
4. *Providers use cached value in adjustment step*

This ensures all providers use the same (possibly stale) distance value, enabling coordinated adjustment.

Proposition 5 (Robustness to Stale Information)**.** *Suppose providers observe stale distances with bounded staleness:*

$$|\Delta_r^\tau(observed) - \Delta_r^\tau(actual)| \leq \varepsilon_{stale}$$

Then the system still converges, with convergence time:

$$T_\varepsilon = O\left(\frac{\log(\varepsilon/\|\Delta_0\|)}{\log(1-\alpha)} + \frac{\varepsilon_{stale}}{\alpha \cdot \min_s q_s}\right)$$

Staleness adds a constant delay but doesn't prevent convergence.

Proof. Stale information causes providers to adjust based on outdated distances. However, the negative feedback mechanism is robust: even with stale Δ_r, providers still move toward targets. The staleness $\varepsilon_{\text{stale}}$ acts as additional noise, requiring extra iterations to overcome. Once staleness resolves (e.g., after network partition heals), normal convergence resumes at rate $(1-\alpha)^t$. □ □

Remark 20 (Eventual Consistency)**.** *The Commons framework naturally supports eventual consistency. Providers can use locally observed distances, and the system self-corrects over time. This makes the system highly resilient to network issues, at the cost of slightly slower convergence during inconsistency periods.*

Definition 22 (Gossip Protocol for Distance Consensus)**.** *An alternative to recipient broadcast is provider gossip:*

1. *Each provider s observes allocations a_{sr}^τ it has made*
2. *Providers periodically gossip their observations to peers*
3. *Each provider estimates: $\hat{\Delta}_r^\tau = |q_r| - \sum_s a_{sr}^\tau$ (based on gossiped info)*
4. *Providers use consensus estimate $\hat{\Delta}_r^\tau$ in adjustments*

This decentralizes distance computation, eliminating single points of failure.

4 Aggregation Along Edges

Definition 23 (Upward Aggregation)**.** *For entity e and type τ:*

$$Agg_{\uparrow}(e, \tau) = \sum_{c \in Categories(e)} \sum_{\substack{p \in P(c) \\ \tau(p) = \tau}} q(p)$$

Definition 24 (Downward Aggregation)**.** *For category c and type τ:*

$$Agg_{\downarrow}(c, \tau) = \sum_{e \in Participants(c)} \sum_{\substack{p \in P(e) \\ \tau(p) = \tau}} q(p)$$

Proposition 6 (Aggregation Duality)**.** *Aggregation preserves the sign structure:*

$$Agg_{\uparrow}(n, \tau) \in \mathbb{R}^{\pm} \tag{8}$$

$$Agg_{\downarrow}(c, \tau) \in \mathbb{R}^{\pm} \tag{9}$$

Proof. Both aggregations are sums of signed quantities $q(p) \in \mathbb{R}^{\pm}$. Sums of signed reals remain in $\mathbb{R}^{\pm}$. □ □

5 Emergent Properties

Theorem 6 (Entity-Type Duality)**.** *Every entity $e \in \mathcal{N}$ can simultaneously be:*

1. *A **type** if $Participants(e) \neq \emptyset$*
2. *An **instance** if $Categories(e) \neq \emptyset$ (i.e., $\Pi(e)(\text{"memberOf"}) \neq \emptyset$)*
3. *A **concrete entity** if $P(e) \neq \emptyset$ (i.e., $\Pi(e)(\text{"potentials"}) \neq \emptyset$)*
4. *A **potential type** if $\exists p \in P(e') : \tau(p) = e$ for some $e' \in \mathcal{N}$*

These roles are not mutually exclusive. Since types are entities ($\tau \in \mathcal{N}$), an entity referenced as a type in a potential can simultaneously be an instance of other types, creating rich type hierarchies.

Proof. The EAV structure makes no distinction between entities serving different roles. An entity may have:

- Other entities with it in their memberOf attribute (making it a type)
- Entities in its own memberOf attribute (making it an instance)
- Values in its potentials attribute (making it concrete)
- Be referenced as τ in other entities' potentials (making it a potential type)

All four roles can coexist. □ □

Theorem 7 (Non-Accumulation)**.** *No node can accumulate beyond its need. Formally, if r receives flow f of type τ and has need $|q_r|$ (where $q_r < 0$), then:*

$$f \leq |q_r|$$

Proof. Immediate from the min function in the Unified Flow Equation. □ □

Theorem 8 (Proportional Fairness)**.** *The allocation ratios equal the weight ratios. For recipients $r_1, r_2 \in R$:*

$$\frac{Flow(s, r_1, \tau)}{Flow(s, r_2, \tau)} = \frac{w(r_1)}{w(r_2)}$$

when neither is need-constrained.

Proof. When $\text{Flow}(s, r_i, \tau) = q_s \cdot \frac{w(r_i)}{\sum_{r' \in R} w(r')}$ for $i = 1, 2$ (i.e., neither hits the min bound), the ratio simplifies to $\frac{w(r_1)}{w(r_2)}$. □ □

Theorem 9 (Determinism)**.** *The allocation function is deterministic: identical inputs produce identical outputs. Formally:*

$$\forall S_1, S_2 : \textit{if } (G_1, P_1, C_1, w_1) = (G_2, P_2, C_2, w_2) \textit{ then } Flow_1 = Flow_2$$

Proof. The Unified Flow Equation is a pure mathematical function with no randomness or hidden state. Each step (constraint checking, weight computation, normalization, allocation) is deterministic. Therefore, identical system states produce identical flows. □ □

6 The Three-Layer Ontology

Definition 25 (Ontological Layers)**.** *The Commons embodies three layers of reality:*

1. ***Structural Layer**: Entity-attribute structure defines what exists through relationships*
2. ***Potential Layer**: Potentials attribute defines what* could *flow through gradients*
3. ***Actual Layer**: Records $\{\Delta_t\}$ define what* did *flow through events*

Remark 21 (Ontological Priority)**.** *The layers exhibit a natural precedence:*

$$\mathcal{N}, \Pi \prec \textit{potentials} \prec \{\Delta_t\}$$

where $\prec$ denotes ontological precedence: entities and their attributes must exist before having potentials, and potentials must exist before being actualized. This is a design principle rather than a mathematical theorem.

7 Composition of Constraints and Weights

Definition 26 (Constraint Algebra)**.** *Constraints form a semilattice under minimum:*

$$(C_1 \wedge C_2)(n) = \min(C_1(n), C_2(n)) \tag{10}$$
$$\top(n) = \infty \quad \textit{(no constraint)} \tag{11}$$
$$\bot(n) = 0 \quad \textit{(total exclusion)} \tag{12}$$

The minimum operation selects the tightest constraint.

Proposition 7 (Filter Algebra as Special Case)**.** *When constraints are restricted to $\{0, \infty\}$, they form a Boolean algebra:*

$$(\Phi_1 \wedge \Phi_2)(n) = \min(\Phi_1(n), \Phi_2(n)) \tag{13}$$
$$(\Phi_1 \vee \Phi_2)(n) = \max(\Phi_1(n), \Phi_2(n)) \tag{14}$$
$$(\neg\Phi)(n) = \begin{cases} \infty & \textit{if } \Phi(n) = 0 \\ 0 & \textit{if } \Phi(n) = \infty \end{cases} \tag{15}$$

This recovers standard Boolean filter composition.

Definition 27 (Constraint Examples)**.** *Common constraint patterns include:*

$$C_{filter}(n) = \begin{cases} \infty & \textit{if eligible}(n) \\ 0 & \textit{otherwise} \end{cases} \quad \textit{(binary)} \tag{16}$$
$$C_{cap}(n) = k \quad \textit{(absolute limit)} \tag{17}$$
$$C_{percent}(n) = \alpha \cdot q_s \quad \textit{(proportional limit)} \tag{18}$$
$$C_{distance}(n) = \max(0, d_{\max} - d(s, n)) \quad \textit{(spatial decay)} \tag{19}$$
$$C_{rate}(n, t) = r \cdot \Delta t \quad \textit{(temporal limit)} \tag{20}$$

Definition 28 (Weight Function)**.** *A **weight function** is $w : N \to \mathbb{R}_{\geq 0}$ assigning preference to each node, subject to normalization:*

$$\sum_{r \in N} w(r) = 1$$

Proposition 8 (Weight Algebra)**.** *Weights form a semiring under:*

$$(w_1 \oplus w_2)(n) = w_1(n) + w_2(n) \tag{21}$$
$$(w_1 \otimes w_2)(n) = w_1(n) \cdot w_2(n) \tag{22}$$

Note that $\otimes$ preserves proportions but requires renormalization: $w' = (w_1 \otimes w_2)/\sum_r (w_1 \otimes w_2)(r)$.

Definition 29 (Weight Examples)**.** *Common weight patterns include:*

***Uniform weights**: $w_{uniform}(n) = 1/|N|$ for all $n \in N$*

Recognition: *Each node n assigns recognition weights $w_n : N \to [0,1]$ with $\sum_r w_n(r) = 1$. The* ***mutual recognition*** *between v_1 and v_2 is:*

$$MR(v_1, v_2) = \min(w_{v_1}(v_2), w_{v_2}(v_1))$$

Normalized MR values into weights:

$$w_{MR}(n) = \frac{MR(p,n)}{\sum_{r \in N} MR(p,r)}$$

Composite weights: *Combine multiple factors through multiplication and renormalization:*

$$w_{composite}(n) \propto w_1(n) \cdot w_2(n) \cdot \ldots \cdot w_k(n)$$

For example, social allocation might weight by mutual recognition and reputation:

$$w_{social}(n) = \frac{MR(provider, n) \cdot reputation(n)}{\sum_{r \in N} MR(provider, r) \cdot reputation(r)}$$

where reputation could be derived from past allocation records or other sources.

Proposition 9 (Composability)**.** *Complex allocation strategies emerge from composition:*

$$C_{complex} = C_1 \wedge C_2 \wedge \cdots \wedge C_n \quad \textit{(tightest constraint)} \tag{23}$$

$$w_{complex} \propto w_1 \otimes w_2 \otimes \cdots \otimes w_n \quad \textit{(multiplicative preference)} \tag{24}$$

8 Dynamic Equilibrium

Definition 30 (Fill Fraction)**.** *For a given allocation round, the* ***fill fraction*** *is:*

$$f = \frac{\sum_{r \in N} \sum_{\tau} Flow(s, r, \tau)}{\sum_{r \in N} \sum_{p \in P(r), q(p) < 0} |q(p)|}$$

This measures what fraction of total need is satisfied per round.

Theorem 10 (Convergence to Equilibrium)**.** *Let $\mathcal{S}^+ = \sum_{s \in N} \sum_{p \in P(s), q(p) > 0} q(p)$ be total capacity and $\mathcal{S}^- = \sum_{r \in N} \sum_{p \in P(r), q(p) < 0} |q(p)|$ be total need.*

Static Case: *If $\mathcal{S}^+ \geq \mathcal{S}^-$ and network state is fixed, then:*

$$\|r(t)\| \leq (1-f)^t \cdot \|r(0)\|$$

where $\|r(t)\|$ is the total unsatisfied need at time t. The system converges to full satisfaction in:

$$T_\varepsilon = O\left(\frac{\log(1/\varepsilon)}{\log(1/(1-f))}\right) \textit{ rounds}$$

Dynamic Case: *When needs and capacities evolve over time, the system does not converge to a fixed point but instead recomputes optimal allocation at each time step based on the current state $S(t) = (G^{(t)}, P^{(t)}, C^{(t)}, w^{(t)})$. The allocation at time t is optimal for the state at time t, providing adaptive rather than convergent behavior.*

Proof. **Static case:** Each round satisfies fraction f of remaining needs. By the allocation capping property, receiving always reduces need. Thus:

$$\|r(t+1)\| = \|r(t) - \phi(r(t))\| \leq (1-f) \cdot \|r(t)\|$$

Iterating gives $\|r(t)\| \leq (1-f)^t \cdot \|r(0)\|$. Setting $(1-f)^t \cdot \|r(0)\| \leq \varepsilon$ and solving for t yields the convergence rate.

Dynamic case: The system recomputes optimal allocation for current state $S(t)$ at each step, maintaining instantaneous optimality without requiring convergence to zero. □ □

Definition 31 (Damping for Stability)**.** *To prevent oscillations in allocation, introduce a damping factor $\delta(r) \in \{0.5, 0.8, 1.0\}$ based on oscillation detection. The **active need** becomes:*

$$q_{active}(r) = q_{declared}(r) \times \delta(r)$$

where $\delta(r)$ decreases when oscillations are detected in r's allocation history.

Proposition 10 (Damping Preserves Convergence)**.** *Damping factors $\delta \in (0, 1]$ reduce the fill fraction but preserve the convergence property:*

$$\|r(t)\| \leq (1-f')^t \cdot \|r(0)\|$$

where $f' = f \cdot \min_r \delta(r) > 0$. The system still converges, though potentially at a slower rate.

Proof. Damping reduces active needs to $q_{\text{active}}(r) = \delta(r) \cdot q_{\text{declared}}(r)$. The fill fraction becomes $f' = f \cdot \min_r \delta(r)$ since less need is expressed. Since $\delta(r) \in (0, 1]$, we have $f' > 0$, and the convergence argument from Theorem 13 applies with f' replacing f. □ □

9 Sovereignty and Anti-Gaming

Remark 22 (Why Normalization Matters)**.** *Weight normalization ($\sum_{r \in N} w(r) = 1$) is essential for meaningful optimization. Without it, entities could assign arbitrarily large weights to all nodes, making $w(r_1)/\sum_{r'} w(r') = 1/n$ independent of preferences. Normalization creates scarcity, forcing genuine trade-offs in preference expression.*

9.1 Weak Monotonicity

Axiom 1 (Weak Monotonicity)**.** *The allocation function satisfies weak monotonicity in weights:*

$$\frac{\partial Flow(s, r, \tau)}{\partial w(r)} \geq 0$$

Increasing a recipient's weight does not decrease their allocation.

Proposition 11 (Monotonicity as Respect)**.** *Weak monotonicity is not merely technical—it is the mathematical expression of respecting sovereign preference expression.*

***If violated**: Increasing weight to beneficial partner r decreases allocation to r, creating perverse incentives to misrepresent preferences.*

***When satisfied**: Honest preference expression is optimal—no strategic manipulation needed.*

9.2 The Anti-Gaming Theorem

We now prove that the Commons framework guarantees anti-gaming properties when weights represent benefit gradients.

Definition 32 (Net-Benefit Gradient)**.** *For entity e with goal G_e, define the* ***net-benefit gradient*** *from provider p:*

$$\beta_e(p) = \frac{\partial \mathbb{P}(G_e)}{\partial \mathit{received}(p, e)} \in \mathbb{R}$$

This measures how much receiving from p contributes to achieving e's goal:

- $\beta_e(p) > 0$*: Receiving from p helps achieve G_e (beneficial)*
- $\beta_e(p) = 0$*: Receiving from p is neutral to G_e (irrelevant)*
- $\beta_e(p) < 0$*: Receiving from p harms achievement of G_e (costly)*

Examples*:*

- ***Hospital****: A patient receiving critical care has $\beta_{hospital}(patient) > 0$ because providing care realizes the hospital's goal of "providing critical care". The patient contributes to goal realization by being a recipient.*
- ***Need-based allocation****: An entity distributing food may set $\beta_e(p) \propto urgency(p)$ or $\beta_e(p) \propto need(p)$, prioritizing those who most need it.*
- ***Harmful relationships****: If receiving from p undermines e's goals (e.g., accepting resources with strings attached that conflict with mission), then $\beta_e(p) < 0$.*

This generalizes beyond private benefit to any *goal-directed allocation, including altruistic, mission-driven, or collective goals.*

Theorem 11 (Anti-Gaming)**.** *Suppose entity e controls weights $w_e : N \to \mathbb{R}_{\geq 0}$ subject to normalization $\sum_r w_e(r) = 1$. If:*

1. *Allocation satisfies the Unified Flow Equation*
2. *Weights satisfy weak monotonicity (Axiom 1)*
3. *Entity seeks to maximize goal achievement:* $\max_{w_e} \mathbb{P}(G_e)$

Then the optimal weight allocation is:

$$w_e^*(r) \propto \max(0, \beta_e(r))$$

Setting weights proportional to positive *net-benefit gradients is optimal.*

Implication*: Honest signaling of relationship benefit is the best strategy. Entities with $\beta_e(r) \leq 0$ should receive zero weight. Gaming (allocating to non-beneficial or harmful partners) is strictly suboptimal.*

Proof. Entity e's goal achievement depends on received allocations:

$$\mathbb{P}(G_e) = f\left(\sum_{p \in N} \beta_e(p) \cdot \text{Flow}(p, e, \tau)\right)$$

From the Unified Flow Equation:

$$\text{Flow}(p, e, \tau) = \min\left(q_p \cdot \frac{w_e(p)}{\sum_{r \in R} w_e(r)}, |q_e|, C(e)\right)$$

In the unconstrained regime (not hitting need limits):

$$\text{Flow}(p, e, \tau) = q_p \cdot \frac{w_e(p)}{\sum_{r \in R} w_e(r)}$$

Substituting:

$$\mathbb{P}(G_e) = f\left(\sum_{p \in N} \beta_e(p) \cdot q_p \cdot \frac{w_e(p)}{\sum_r w_e(r)}\right)$$

With normalization $\sum_r w_e(r) = 1$:

$$\mathbb{P}(G_e) = f\left(\sum_{p \in N} \beta_e(p) \cdot q_p \cdot w_e(p)\right)$$

To maximize, take derivative with respect to $w_e(p)$ subject to $\sum_r w_e(r) = 1$:

$$\mathcal{L} = \sum_p \beta_e(p) \cdot q_p \cdot w_e(p) - \lambda\left(\sum_r w_e(r) - 1\right)$$

First-order conditions:

$$\frac{\partial \mathcal{L}}{\partial w_e(p)} = \beta_e(p) \cdot q_p - \lambda = 0$$

Therefore:

$$w_e^*(p) \propto \beta_e(p) \cdot q_p$$

Conclusion: Optimal weights are proportional to the product of net-benefit gradients and provider capacities: $w_e^*(p) \propto \max(0, \beta_e(p) \cdot q_p)$. This naturally accounts for varying provider capacities: larger capacity providers receive proportionally more weight. Allocating weight to neutral partners ($\beta_e(p) = 0$) or harmful partners ($\beta_e(p) < 0$) is strictly suboptimal regardless of their capacity. □ □

Remark 23 (Capacity-Aware Weighting Emerges Naturally)**.** *Recipients don't need explicit knowledge of provider capacities. The iterative convergence process naturally teaches them: providers with larger capacity consistently deliver more, leading recipients to increase their expectations from those providers over time. This is a form of* ***learned prioritization*** *where the system discovers optimal weights through experience, not through explicit capacity information exchange.*

Corollary 3 (Honest Signaling)**.** *Under the Commons framework:*

- ***Beneficial partners****: Allocate weight proportional to benefit → receive more*
- ***Non-beneficial partners****: Allocate zero weight → receive nothing*
- ***Gaming (false signaling)****: Allocate weight to non-beneficial partners → reduces allocation from beneficial ones → decreases goal achievement*

Honest preference expression is optimal.

Corollary 4 (Sybil Resistance)**.** *Creating fake identities (Sybils) provides no advantage:*
Proof*: If entity e creates Sybil s and allocates weight $w_e(s) > 0$:*

- *Sybil s has zero capacity ($q_s = 0$) or provides no benefit ($\beta_e(s) = 0$)*
- *By anti-gaming theorem, $w_e^*(s) = 0$*
- *Allocating $w_e(s) > 0$ reduces weight available for beneficial partners*
- *Result: Decreases goal achievement*

Sybil creation is strictly suboptimal. □

9.3 Controllability and Memory

Proposition 12 (Controllability)**.** *The Commons framework permits memory in weight functions, provided current weights retain ultimate control.*
Valid (memory with controllability)*:*

$$w^{(t)}(r) = \alpha \cdot w^{(t)}_{current}(r) + \beta \cdot w^{(t-1)}_{past}(r) \quad \text{with } \alpha > 0$$

Current weights can override past patterns.
Invalid (memory without controllability)*:*

$$w^{(t)}(r) = \begin{cases} w^{(t)}_{current}(r) & \text{if never decreased in past} \\ 0 & \text{if ever decreased in past} \end{cases}$$

Past actions create dead zones—violates sovereignty.
Key insight*: Learning, adaptation, and temporal smoothing are permitted as long as current weights can achieve any desired allocation.*

10 The Paradigm Shift

Proposition 13 (Unification)**.** *The Commons framework provides a unified treatment of:*

1. ***Categorization*** *(memberOf attribute creating emergent graph)*
2. ***Resource allocation*** *(potentials attribute enabling flow equilibration)*
3. ***Type inheritance*** *(upward aggregation along memberOf)*

4. ***Computation*** *(constraint satisfaction + preference optimization)*

Each emerges as a pattern within the Entity-Attribute-Value model.

Proof.

- Categorization: memberOf attribute creates type hierarchies (Definition 5)
- Allocation: The Unified Flow Equation distributes potentials (Theorem 1)
- Inheritance: $\text{Agg}_{\uparrow}$ propagates along memberOf (Definition 11)
- Computation: Constraints C limit possibilities, weights w optimize preferences (Section 6)

Each is expressible within $\mathcal{C} = \langle \mathcal{N}, \Pi, \{\Delta_t\} \rangle$. □ □

11 A Simple Example

Consider a kitchen k with meal potential $(\texttt{meals}, +100)$ and two people: Alice with potential $(\texttt{meals}, -20)$ and Bob with potential $(\texttt{meals}, -30)$. Using unit weights ($w(n) = 1$ for all n), the Unified Flow Equation gives:

$$\text{Flow}(k, \text{Alice}, \texttt{meals}) = \min\left(100 \cdot \frac{1}{1+1}, 20\right) = \min(50, 20) = 20 \tag{25}$$

$$\text{Flow}(k, \text{Bob}, \texttt{meals}) = \min\left(100 \cdot \frac{1}{1+1}, 30\right) = \min(50, 30) = 30 \tag{26}$$

Alice receives 20 meals, Bob receives 30 meals, in exact proportion to their needs. The allocation is automatic, fair, and complete.

12 Conclusion

Five primitives—graphs, potentials, constraints, weights, and records—yield one equation. From this equation emerges a remarkable property: when weights represent benefit gradients, honest preference signaling is the unique optimal strategy. Strategic manipulation, vote trading, coalition formation, and Sybil attacks all become strictly suboptimal.

This provides mathematical foundations for sovereign coordination: systems where participants retain full autonomy over their preferences while achieving collectively optimal resource allocation. Categorization, allocation, and type hierarchies all emerge as patterns within the same unified structure.